UNIT 1

Activity: Top 5 Cities by Average Revenue per Category

CO Mapped: CO1 –Use Big Data and its Business Implications.

Type: Practical + Analytical Activity

Problem Statement

In large-scale e-commerce systems, businesses operate across multiple cities and product categories. Each transaction generates revenue based on the quantity of items sold and their price. To analyze business performance, it is important to determine which cities generate the highest average revenue within each product category.Given a large sales dataset containing transaction details such as

Category, City, Quantity, and Price, develop a MapReduce program to:

1. Calculate the average revenue for each (Category, City) pair.

2. Identify the Top 5 cities with the highest average revenue in each category.

Sample Input Dataset

e-commerce_transaction.csv

TransactionID,UserID,ProductID,Category,City,Quantity,Price,PaymentMode,Date

T1001,U101,P501,Electronics,Mumbai,1,25000,CreditCard,2026-02-14

T1002,U102,P502,Fashion,Delhi,3,1500,UPI,2026-02-15

T1003,U103,P503,Grocery,Bangalore,5,200,Cash,2026-02-16

T1004,U104,P504,Electronics,Chennai,2,18000,DebitCard,2026-02-17

T1005,U105,P505,Fashion,Kolkata,4,2200,UPI,2026-02-18

T1006,U106,P506,Electronics,Hyderabad,1,32000,CreditCard,2026-02-18

Objective

This activity focuses on implementing a multi-stage MapReduce solution to compute the Top 5 Cities by Average Revenue per Category from a sales dataset. Through this activity, student able to compute distributed averages, apply reducer-side sorting, and extract Top-K results using Hadoop MapReduce.

```
hduser@nitttr-HP-Elite-Tower-800-G9-Desktop-PC:~/BDT_Activity2$ hdfs dfs -ls /input_reve
nue
2026-03-10 15:29:04,157 WARN util.NativeCodeLoader: Unable to load native-hadoop library
 for your platform... using builtin-java classes where applicable
Found 1 items
-rw-r--r--   1 hduser supergroup       5895 2026-03-10 14:58 /input_revenue/e_commerce_t
ransaction.csv
```

hdfs dfs -mkdir /input

hdfs dfs -put ecommerce_transactions.csv /input

hdfs dfs -ls /input

GitHub link: https://github.com/nss477/Big_Data_Tools_activity2

Top 5 Cities by Average Revenue per Category using Hadoop MapReduce

1. Introduction

In modern e-commerce systems, millions of transactions occur daily across different cities and product categories. Analyzing this data helps businesses identify high-performing markets and optimize their sales strategies. Big Data technologies such as Hadoop enable organizations to process large datasets efficiently using distributed computing.

The objective of this activity is to analyze an e-commerce transaction dataset and identify the **Top 5 cities generating the highest average revenue for each product category**. The dataset contains fields such as TransactionID, Category, City, Quantity, and Price. Hadoop MapReduce is used to process this dataset in a distributed manner and compute the required results.

The solution is implemented using a **two-stage MapReduce process**. The first job calculates the average revenue for each (Category, City) pair. The second job groups cities by category and extracts the top five cities based on average revenue.

2. Dataset Description

The dataset used in this activity is stored in CSV format and contains the following attributes:

Field	Description
TransactionID	Unique transaction identifier

UserID	Identifier of the customer
ProductID	Unique product identifier
Category	Product category (Electronics, Fashion, Grocery)
City	City where transaction occurred
Quantity	Number of items purchased
Price	Price per item
PaymentMode	Payment method used
Date	Transaction date

Example dataset record:

T1001,U101,P501,Electronics,Mumbai,1,25000,CreditCard,2026-02-14
T1002,U102,P502,Fashion,Delhi,3,1500,UPI,2026-02-15
T1003,U103,P503,Grocery,Bangalore,5,200,Cash,2026-02-16

3. Data Preprocessing

Before processing the dataset with MapReduce, several preprocessing steps were performed:

1. The dataset was uploaded to the Hadoop Distributed File System (HDFS).
2. The header row was ignored during processing because it does not represent transaction data.
3. Only relevant fields were extracted: **Category, City, Quantity, and Price**.
4. Revenue for each transaction was calculated using the formula:

Revenue = Quantity × Price

This revenue value is used as the primary metric for further analysis.

4. MapReduce Job 1 – Average Revenue Calculation

```
javac -classpath `hadoop classpath` -d . *.java

jar -cvf ecommerce.jar *
```

```
hadoop jar ecommerce.jar Driver1 /input /output2

hdfs dfs  -cat /output2/part-r-00000
```

Mapper Logic

The Mapper reads the dataset line by line and performs the following operations:

1. Skip the header row.
2. Split the CSV record using a comma delimiter.
3. Extract Category, City, Quantity, and Price.
4. Calculate revenue using:

Revenue = Quantity × Price

5. Emit the key-value pair:

Key → Category|City
Value → Revenue

Example Mapper output:

Electronics|Mumbai → 25000
Electronics|Chennai → 36000
Fashion|Delhi → 4500

These outputs are automatically grouped by Hadoop during the shuffle phase.

Reducer Logic

The Reducer receives a key representing **Category|City** and a list of revenue values.

Reducer operations:

1. Sum all revenue values.
2. Count the number of transactions.
3. Compute the average revenue:

Average Revenue = Total Revenue / Transaction Count

Reducer output format:

Category|City → AverageRevenue

Example intermediate output:

Electronics|Delhi→66000

Electronics|Mumbai→42714

Fashion|Chennai→7983

```
hduser@nitttr-HP-Elite-Tower-800-G9-Desktop-PC:~/BDT_Activity2$ hadoop jar ecommerce.jar Driver1 /input_revenue /output2
2026-03-10 15:16:35,713 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-03-10 15:16:36,052 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at localhost/127.0.0.1:8032
2026-03-10 15:16:36,222 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner
to remedy this.
2026-03-10 15:16:36,236 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/hduser/.staging/job_1773134794273_0002
2026-03-10 15:16:36,378 INFO input.FileInputFormat: Total input files to process : 1
2026-03-10 15:16:36,413 INFO mapreduce.JobSubmitter: number of splits:1
2026-03-10 15:16:36,485 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1773134794273_0002
2026-03-10 15:16:36,485 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-03-10 15:16:36,579 INFO conf.Configuration: resource-types.xml not found
2026-03-10 15:16:36,579 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-03-10 15:16:36,712 INFO impl.YarnClientImpl: Submitted application application_1773134794273_0002
2026-03-10 15:16:36,734 INFO mapreduce.Job: The url to track the job: http://nitttr-HP-Elite-Tower-800-G9-Desktop-PC:8088/proxy/application_1773134794273_0002/
2026-03-10 15:16:36,734 INFO mapreduce.Job: Running job: job_1773134794273_0002
2026-03-10 15:16:41,799 INFO mapreduce.Job: Job job_1773134794273_0002 running in uber mode : false
2026-03-10 15:16:41,801 INFO mapreduce.Job:  map 0% reduce 0%
2026-03-10 15:16:44,873 INFO mapreduce.Job:  map 100% reduce 0%
2026-03-10 15:16:48,912 INFO mapreduce.Job:  map 100% reduce 100%
2026-03-10 15:16:48,924 INFO mapreduce.Job: Job job_1773134794273_0002 completed successfully
2026-03-10 15:16:48,999 INFO mapreduce.Job: Counters: 54
        File System Counters
                FILE: Number of bytes read=2755
                FILE: Number of bytes written=557591
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=6022
                HDFS: Number of bytes written=560
                HDFS: Number of read operations=8
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=2
                HDFS: Number of bytes read erasure-coded=0
        Job Counters
                Launched map tasks=1
                Launched reduce tasks=1
                Data-local map tasks=1
                Total time spent by all maps in occupied slots (ms)=1072
                Total time spent by all reduces in occupied slots (ms)=1207
                Total time spent by all map tasks (ms)=1072
                Total time spent by all reduce tasks (ms)=1207
                Total vcore-milliseconds taken by all map tasks=1072
                Total vcore-milliseconds taken by all reduce tasks=1207
```

```
                Total time spent by all reduces in occupied slots (ms)=1207
                Total time spent by all map tasks (ms)=1072
                Total time spent by all reduce tasks (ms)=1207
                Total vcore-milliseconds taken by all map tasks=1072
                Total vcore-milliseconds taken by all reduce tasks=1207
                Total megabyte-milliseconds taken by all map tasks=1097728
                Total megabyte-milliseconds taken by all reduce tasks=1235968
        Map-Reduce Framework
                Map input records=101
                Map output records=100
                Map output bytes=2549
                Map output materialized bytes=2755
                Input split bytes=127
                Combine input records=0
                Combine output records=0
                Reduce input groups=18
                Reduce shuffle bytes=2755
                Reduce input records=100
                Reduce output records=18
                Spilled Records=200
                Shuffled Maps =1
                Failed Shuffles=0
                Merged Map outputs=1
                GC time elapsed (ms)=58
                CPU time spent (ms)=580
                Physical memory (bytes) snapshot=632561664
                Virtual memory (bytes) snapshot=5260824576
                Total committed heap usage (bytes)=926416896
                Peak Map Physical memory (bytes)=296017920
                Peak Map Virtual memory (bytes)=2627575808
                Peak Reduce Physical memory (bytes)=336543744
                Peak Reduce Virtual memory (bytes)=2633248768
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=5895
        File Output Format Counters
                Bytes Written=560
hduser@nitttr-HP-Elite-Tower-800-G9-Desktop-PC:~/BDT_Activity2$ hadoop jar ecommerce.jar Driver2 /input_revenue /output3
```

```
hduser@nitttr-HP-Elite-Tower-800-G9-Desktop-PC:~/BDT_Activity2$ hdfs dfs -cat /input_revenue/e_commerce_transaction.csv | head
2026-03-10 15:24:37,225 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
TransactionID,UserID,ProductID,Category,City,Quantity,Price,PaymentMode,Date
T1001,U101,P501,Electronics,Mumbai,1,25000,CreditCard,2026-02-01
T1002,U102,P502,Fashion,Delhi,3,1500,UPI,2026-02-02
T1003,U103,P503,Grocery,Bangalore,5,200,Cash,2026-02-02
T1004,U104,P504,Electronics,Chennai,2,18000,DebitCard,2026-02-03
T1005,U105,P505,Fashion,Kolkata,4,2200,UPI,2026-02-03
T1006,U106,P506,Electronics,Hyderabad,1,32000,CreditCard,2026-02-04
T1007,U107,P507,Grocery,Mumbai,6,150,Cash,2026-02-04
T1008,U108,P508,Fashion,Delhi,2,3000,DebitCard,2026-02-05
T1009,U109,P509,Electronics,Bangalore,1,40000,UPI,2026-02-05
hduser@nitttr-HP-Elite-Tower-800-G9-Desktop-PC:~/BDT_Activity2$ hdfs dfs -cat /output2/part-r-00000
2026-03-10 15:25:09,667 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Electronics|Bangalore   40750.0
Electronics|Chennai     47000.0
Electronics|Delhi       66000.0
Electronics|Hyderabad   46142.857142857145
Electronics|Kolkata     58600.0
Electronics|Mumbai      42714.28571428572
Fashion|Bangalore       7266.666666666667
Fashion|Chennai 7983.333333333333
Fashion|Delhi   6800.0
Fashion|Hyderabad       6833.333333333333
Fashion|Kolkata 7150.0
Fashion|Mumbai  7771.428571428572
Grocery|Bangalore       1105.625
Grocery|Chennai 1238.3333333333333
Grocery|Delhi   1297.142857142857
Grocery|Hyderabad       1140.0
Grocery|Kolkata 1177.857142857143
Grocery|Mumbai  1011.6666666666666
```

5. MapReduce Job 2 – Top 5 Cities per Category

hadoop jar ecommerce.jar Driver2 /output2 /output3

hdfs dfs -cat /output3/part-r-00000

Mapper Logic

The second mapper reorganizes the intermediate output from Job 1.

Input format:

Category|City → AverageRevenue

Mapper splits the key and emits:

Key → Category
Value → City:AverageRevenue

Example output:

Electronics → Mumbai:42714
Electronics → Delhi:66000
Fashion → Chennai:7983

Reducer Logic

The Reducer receives all cities belonging to the same category.

Reducer steps:

1. Store city-revenue pairs in a list.
2. Sort the list in **descending order of revenue**.
3. Select the **Top 5 cities**.
4. Output the ranked results.

Example final output:

Electronics Delhi:66000
Electronics Kolkata:58600
Electronics Hyderabad:46142
Electronics Mumbai:42714
Electronics Bangalore:40750

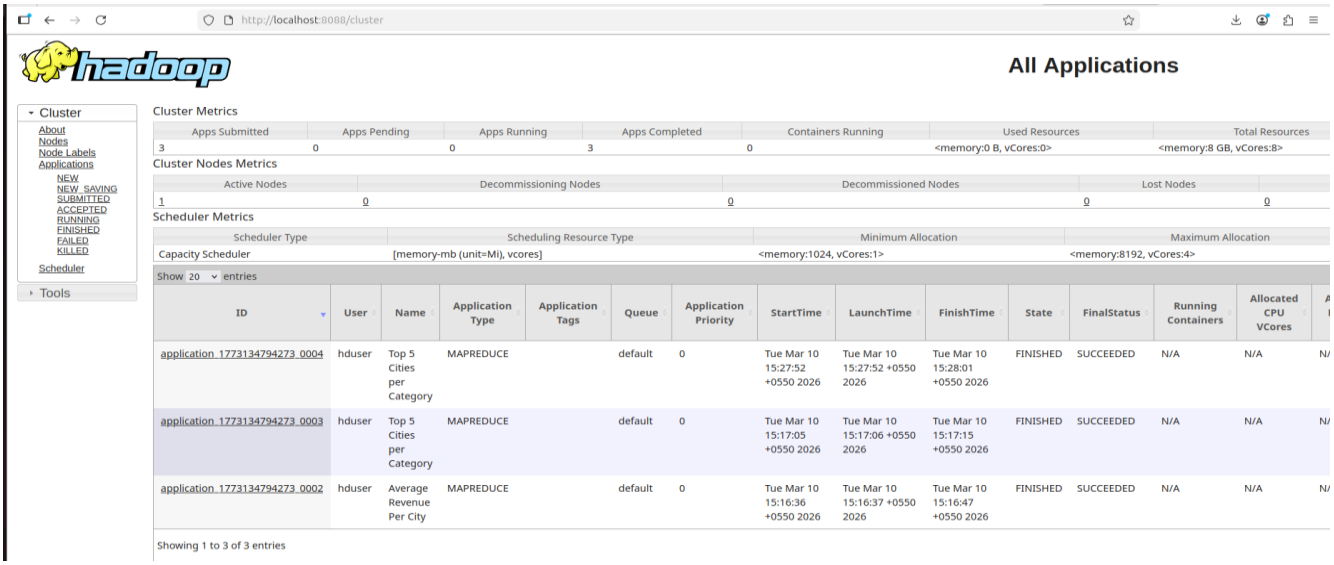
Fashion Chennai:7983
Fashion Mumbai:7771
Fashion Bangalore:7266
Fashion Kolkata:7150
Fashion Hyderabad:6833

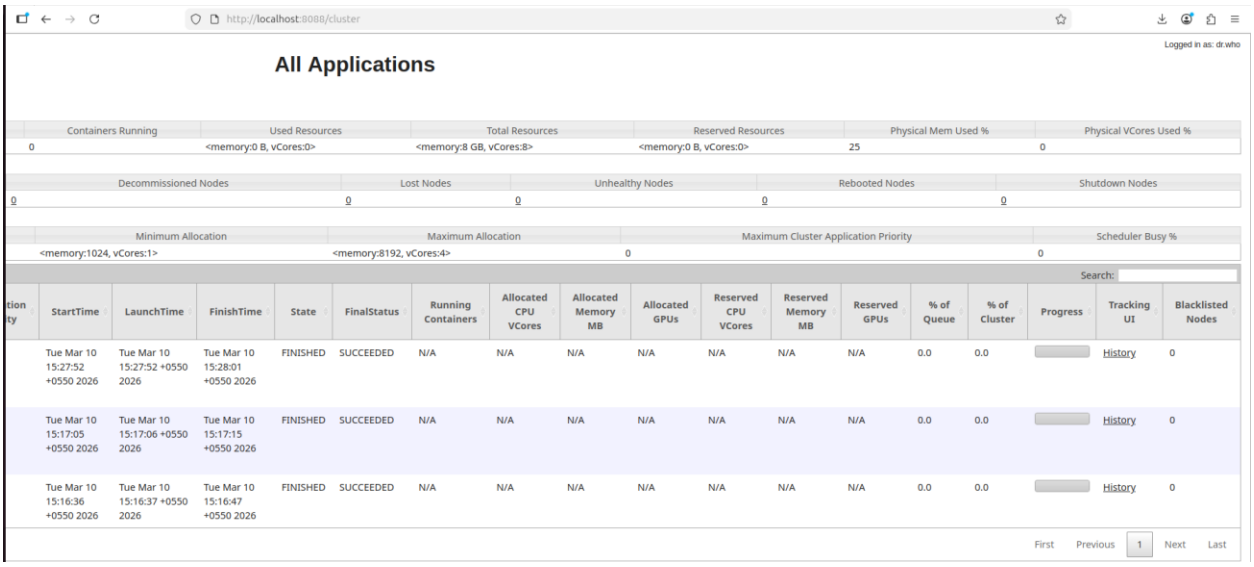
Grocery Delhi:1297
Grocery Chennai:1238
Grocery Kolkata:1177
Grocery Hyderabad:1140
Grocery Bangalore:1105

```
hduser@nitttr-HP-Elite-Tower-800-G9-Desktop-PC:~/BDT_Activity2$ hadoop jar ecommerce.jar Driver2 /input_revenue /output3
2026-03-10 15:17:04,576 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-03-10 15:17:04,913 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at localhost/127.0.0.1:8032
2026-03-10 15:17:05,088 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner
to remedy this.
2026-03-10 15:17:05,100 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/hduser/.staging/job_1773134794273_0003
2026-03-10 15:17:05,236 INFO input.FileInputFormat: Total input files to process : 1
2026-03-10 15:17:05,274 INFO mapreduce.JobSubmitter: number of splits:1
2026-03-10 15:17:05,342 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1773134794273_0003
2026-03-10 15:17:05,342 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-03-10 15:17:05,431 INFO conf.Configuration: resource-types.xml not found
2026-03-10 15:17:05,431 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-03-10 15:17:05,464 INFO impl.YarnClientImpl: Submitted application application_1773134794273_0003
2026-03-10 15:17:05,485 INFO mapreduce.Job: The url to track the job: http://nitttr-HP-Elite-Tower-800-G9-Desktop-PC:8088/proxy/application_1773134794273_0003/
2026-03-10 15:17:05,485 INFO mapreduce.Job: Running job: job_1773134794273_0003
2026-03-10 15:17:09,527 INFO mapreduce.Job: Job job_1773134794273_0003 running in uber mode : false
2026-03-10 15:17:09,527 INFO mapreduce.Job:  map 0% reduce 0%
2026-03-10 15:17:13,572 INFO mapreduce.Job:  map 100% reduce 0%
2026-03-10 15:17:16,589 INFO mapreduce.Job:  map 100% reduce 100%
2026-03-10 15:17:17,617 INFO mapreduce.Job: Job job_1773134794273_0003 completed successfully
2026-03-10 15:17:17,683 INFO mapreduce.Job: Counters: 54
        File System Counters
                FILE: Number of bytes read=6
                FILE: Number of bytes written=552083
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=6022
                HDFS: Number of bytes written=0
                HDFS: Number of read operations=8
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=2
                HDFS: Number of bytes read erasure-coded=0
        Job Counters
                Launched map tasks=1
                Launched reduce tasks=1
                Data-local map tasks=1
                Total time spent by all maps in occupied slots (ms)=1070
                Total time spent by all reduces in occupied slots (ms)=1101
                Total time spent by all map tasks (ms)=1070
                Total time spent by all reduce tasks (ms)=1101
                Total vcore-milliseconds taken by all map tasks=1070
```

```
                Total time spent by all maps in occupied slots (ms)=1070
                Total time spent by all reduces in occupied slots (ms)=1101
                Total time spent by all map tasks (ms)=1070
                Total time spent by all reduce tasks (ms)=1101
                Total vcore-milliseconds taken by all map tasks=1070
                Total vcore-milliseconds taken by all reduce tasks=1101
                Total megabyte-milliseconds taken by all map tasks=1095680
                Total megabyte-milliseconds taken by all reduce tasks=1127424
        Map-Reduce Framework
                Map input records=101
                Map output records=0
                Map output bytes=0
                Map output materialized bytes=6
                Input split bytes=127
                Combine input records=0
                Combine output records=0
                Reduce input groups=0
                Reduce shuffle bytes=6
                Reduce input records=0
                Reduce output records=0
                Spilled Records=0
                Shuffled Maps =1
                Failed Shuffles=0
                Merged Map outputs=1
                GC time elapsed (ms)=55
                CPU time spent (ms)=470
                Physical memory (bytes) snapshot=614514688
                Virtual memory (bytes) snapshot=5253750784
                Total committed heap usage (bytes)=922746880
                Peak Map Physical memory (bytes)=296521728
                Peak Map Virtual memory (bytes)=2623922176
                Peak Reduce Physical memory (bytes)=317992960
                Peak Reduce Virtual memory (bytes)=2629828608
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=5895
        File Output Format Counters
                Bytes Written=0
```

```
hduser@nitttr-HP-Elite-Tower-800-G9-Desktop-PC:~/BDT_Activity2$ hdfs dfs -cat /output3/part-r-00000
2026-03-10 15:28:07,476 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Electronics     Delhi:66000.0
Electronics     Kolkata:58600.0
Electronics     Chennai:47000.0
Electronics     Hyderabad:46142.857142857145
Electronics     Mumbai:42714.28571428572
Fashion Chennai:7983.333333333333
Fashion Mumbai:7771.428571428572
Fashion Bangalore:7266.666666666667
Fashion Kolkata:7150.0
Fashion Hyderabad:6833.333333333333
Grocery Delhi:1297.142857142857
Grocery Chennai:1238.3333333333333
Grocery Kolkata:1177.857142857143
Grocery Hyderabad:1140.0
Grocery Bangalore:1105.625
```





6. Challenges Faced

Several challenges were encountered during implementation:

1. **Handling CSV parsing errors** when records contained unexpected formatting.
2. Ensuring the **header row was skipped** to avoid incorrect calculations.
3. Correctly splitting intermediate data using **tab delimiters between key and value**.
4. Implementing **sorting logic inside the reducer** to rank cities by revenue.
5. Managing Hadoop commands, jar creation, and HDFS directories without path errors.

These challenges were resolved through debugging, verifying intermediate outputs, and testing each MapReduce stage separately.

7. Why Combiner Was Not Used

A **Combiner** is used in MapReduce to perform local aggregation at the mapper side to reduce network traffic before sending data to reducers.

In this project, a combiner was **not used** because:

1. The program requires **accurate average revenue calculation**, which depends on both **sum and count** of values.
2. The reducer logic calculates **average using total revenue and transaction count**.
3. If a combiner is used incorrectly, it may produce **incorrect averages**, because averaging intermediate values does not guarantee the correct global average.
4. Additionally, the second MapReduce job performs **sorting and ranking**, which cannot be done by a combiner.

Therefore, the combiner was avoided to maintain **accuracy and correctness of results**.

6. Performance Comparison (With vs Without Combiner)

Feature	Without Combiner	With Combiner
Network Traffic	Higher	Lower
Mapper Output Data	Large	Reduced
Accuracy for Average	Correct	Risk of incorrect results
Suitability for Ranking	Works correctly	Not applicable

Explanation:

- Using a combiner would reduce the amount of data transferred during the shuffle phase.
- However, because the program calculates averages and performs ranking operations, using a combiner may lead to **incorrect aggregation results**.
- Therefore, the system prioritizes **correctness over minor performance gains**.

7. Conclusion

This activity demonstrated how Hadoop MapReduce can be used to process large datasets efficiently. The program successfully calculated the average revenue for each city-category pair and identified the top five cities with the highest revenue in each category.

The use of distributed computing allowed the dataset to be processed in parallel, making the system scalable for large e-commerce platforms. This analysis can help businesses identify profitable markets and make data-driven decisions to improve sales strategies.